

Deployment Guide

Self-hosted PWA at fms.argontechs.dev — CloudPanel + PM2 on a VPS

Stack	Nuxt 4 + Nitro (node-server) · SQLite WAL · Drizzle · PM2 fork
Reverse proxy	CloudPanel nginx + Let's Encrypt (TLS 443 → 127.0.0.1:3000)
Runtime	Node.js 22+ · single-user · single instance
Database	better-sqlite3 WAL · /home/fms/data/money.sqlite
Site user / home	fms · /home/fms
Document date	2026-06-19

1 Overview & Architecture

Personal-FMS is a **single-user PWA** for personal finance habit-building. It tracks cash flow, safe-to-spend, debt payoff, and drives push reminders via an in-process scheduler. Everything runs in one process on one VPS — no managed database, no serverless, no second daemon.

```
Internet (HTTPS 443) | ▼ nginx — CloudPanel vhost (Let's Encrypt TLS, HTTP→HTTPS
redirect, | X-Forwarded-Proto, Upgrade headers, dotfile deny, | location /api/internal/ {
deny all }) | ▼ 127.0.0.1:3000 | ▼ PM2 — process: money-fms (fork mode, 1 instance) |
script: .output/server/index.mjs | cwd: /home/fms/htdocs/fms.argontechs.dev | env:
/home/fms/htdocs/fms.argontechs.dev/.env | — Nuxt 4 Vue SPA (service worker, offline
quick-log, push subscribe) | — Nitro server routes (server/api/**) | requireSession()
gates every route except /api/auth/login | — In-process Nitro scheduledTasks (croner) |
notify-dispatch */5 * * * * (bill-due + payday-save prompts) | post-recurring 5 0 * * *
(auto-post salary/bills/interest) | — SQLite WAL —> /home/fms/data/money.sqlite OS-cron
(fms user, TZ=Asia/Kuala_Lumpur) — */5 * * * * run-due watchdog → POST
127.0.0.1:3000/api/internal/run-due — 15 3 * * * backup-db.sh → .backup → gzip → rclone
→ gdrive-money: — 45 3 * * * restore-verify.sh → integrity_check + row-count + freshness
```

The Nitro in-process scheduler is the **primary** dispatch path. The OS-cron watchdog is **permanent insurance** — both fire on the same 5-minute grid; the `notifications_sent UNIQUE(kind, ref_id, scheduled_for)` constraint guarantees exactly-once delivery even when both paths fire in the same window.

2 Prerequisites

Requirement	Detail	Status
CloudPanel VPS	Any Linux VPS with CloudPanel already installed. The user already runs PWA-PropertyAgentCRM on this box — same provisioning path.	READY
Node.js 22+	CloudPanel's Node.js site uses Node 22. Verify with <code>node --version</code> after site creation.	READY
DNS A record	<code>fms.argontechs.dev</code> A record already points to the VPS public IPv4. Confirm: <code>dig +short fms.argontechs.dev</code> returns the VPS IP before the cert step — Let's Encrypt HTTP-01 fails otherwise.	DONE
SSH access	Root access for site creation and linger; site-user <code>fms</code> SSH for code + cron.	
rclone	Installed on the VPS for Google Drive backups. <code>apt install rclone</code> or binary from rclone.org .	
sqlite3 CLI	For backup and verification commands. <code>apt install sqlite3</code> .	

CONFIRM SITE-USER NAMING FIRST

Run `ls /home` and `clpctl site:list` on the VPS to see how CloudPanel named existing sites (e.g. `propertycrm`). This guide uses site user `fms` throughout. Use whatever CloudPanel creates uniformly for SSH, docroot, crontab, and `pm2 startup` — do not mix forms.

3 Create the Node.js Site in CloudPanel

Run as **root** on the VPS. This creates the Linux site user, docroot, and nginx vhost that reverse-proxies all traffic to `127.0.0.1:3000`.

```
# Generate a strong password once, store it in your password manager immediately
SITE_PW="$(openssl rand -base64 24)"
echo "Site password: $SITE_PW" # copy this NOW — it won't be shown again

clpctl site:add:nodejs \
  --domainName=fms.argontechs.dev \
  --nodejsVersion=22 \
  --appPort=3000 \
  --siteUser=fms \
  --siteUserPassword="$SITE_PW"

unset SITE_PW
```

Verify the site was created:

```
# Docroot exists and is owned by the site user
ls -ld /home/fms/htdocs/fms.argontechs.dev
stat -c '%U:%G' /home/fms/htdocs/fms.argontechs.dev
# Expected: fms:fms

# Nginx vhost targets the loopback port
grep -Rn '127.0.0.1:3000' /etc/nginx/
# Expected: a proxy_pass http://127.0.0.1:3000; line
```

CloudPanel automatically writes an nginx vhost that terminates TLS (added in §8) and reverse-proxies to Node. Node is never exposed directly to the internet.

4 Clone the Code

SSH as the **site user** (not root) for all remaining steps unless noted.

```
ssh fms@fms.argontechs.dev

cd /home/fms/htdocs/fms.argontechs.dev

# Clone into the existing docroot (note the trailing dot)
git clone https://github.com/<owner>/personal-fms.git .
# or via SSH:
# git clone git@github.com:<owner>/personal-fms.git .

# Install dependencies (production-exact versions from lockfile)
npm ci
```

Verify:

```
git rev-parse --short HEAD      # prints a commit SHA
ls package.json ecosystem.config.cjs # both present
node --version                  # v22.x.x
```

5 Environment Variables

5.1 Generate secrets (run once)

```
# Generate VAPID keypair (do NOT rotate casually – rotation
# invalidates every push subscription)
npx web-push generate-vapid-keys
# Output:
#   Public Key: BAxxx...
#   Private Key: yyy...
# Copy both values – you need them below.

# Session password (seals the httpOnly cookie)
openssl rand -hex 32  # → 64 hex chars

# Watchdog secret (guards /api/internal/run-due)
openssl rand -hex 16  # → 32 hex chars
```

5.2 Write .env

NEVER COMMIT .ENV

The `.env` file contains private keys and must stay on the box only. The repo's `.gitignore` covers `.env` — verify before any push.

```

cat > /home/fms/htdocs/fms.argontechs.dev/.env << 'EOF'
NODE_ENV=production
TZ=Asia/Kuala_Lumpur
NITRO_HOST=127.0.0.1
NITRO_PORT=3000
DATABASE_URL=file:/home/fms/data/money.sqlite

# VAPID – Web Push keys (generate once with: npx web-push generate-vapid-keys)
# The public key is needed by BOTH server (signing) and client (subscribe)
Nuxt_PUBLIC_VAPID_PUBLIC_KEY=<paste Public Key from web-push output>
Nuxt_VAPID_PRIVATE_KEY=<paste Private Key from web-push output>
Nuxt_VAPID_SUBJECT=mailto:yongwei1127@gmail.com

# Session sealing key – seals the httpOnly cookie (≥32 chars)
Nuxt_SESSION_PASSWORD=<openssl rand -hex 32>

# Watchdog secret – guards /api/internal/run-due (constant-time compare)
Nuxt_RUN_DUE_SECRET=<openssl rand -hex 16>

# Optional: SMTP for email fallback (payday prompts if push unavailable)
# Nuxt_SMTP_URL=smtp://user:pass@host:587

# One-time user bootstrap (remove after running seed:user)
SEED_USERNAME=<your-login-username>
SEED_PASSWORD=<your-login-password>
EOF

# Lock permissions – readable only by the site user
chmod 600 /home/fms/htdocs/fms.argontechs.dev/.env

```

Verify:

```

stat -c '%a %U' /home/fms/htdocs/fms.argontechs.dev/.env
# Expected: 600 fms

# Check both VAPID keys are present and non-empty
set -a; . ./env; set +a
test -n "$Nuxt_PUBLIC_VAPID_PUBLIC_KEY" && echo "public key OK"
test -n "$Nuxt_VAPID_PRIVATE_KEY" && echo "private key OK"
test -n "$Nuxt_SESSION_PASSWORD" && echo "session password OK"
test -n "$Nuxt_RUN_DUE_SECRET" && echo "watchdog secret OK"

```

5.3 Full environment variable reference

Variable	Value / how to generate	Notes
<code>NODE_ENV</code>	<code>production</code>	Required
<code>TZ</code>	<code>Asia/Kuala_Lumpur</code>	Mandatory — scheduler gates on MYT 09:00
<code>NITRO_HOST</code>	<code>127.0.0.1</code>	Bind to loopback only
<code>NITRO_PORT</code>	<code>3000</code>	Must match <code>--appPort</code> in <code>clpctl</code>
<code>DATABASE_URL</code>	<code>file:/home/fms/data/money.sqlite</code>	Outside docroot — survives redeloys
<code>NEXT_PUBLIC_VAPID_PUBLIC_KEY</code>	<code>npx web-push generate-vapid-keys</code>	Client-exposed; used by <code>pushManager.subscribe()</code>
<code>NEXT_VAPID_PRIVATE_KEY</code>	Same keygen command	Server-only; signs push payloads
<code>NEXT_VAPID_SUBJECT</code>	<code>mailto:yongwei1127@gmail.com</code>	Contact for push providers
<code>NEXT_SESSION_PASSWORD</code>	<code>openssl rand -hex 32</code>	≥32 chars; seals <code>httpOnly</code> session cookie
<code>NEXT_RUN_DUE_SECRET</code>	<code>openssl rand -hex 16</code>	Guards <code>/api/internal/run-due</code>
<code>NEXT_SMTP_URL</code>	<code>smtp://user:pass@host:587</code>	Optional email fallback channel
<code>SEED_USERNAME</code>	Your login username	One-time; remove after <code>seed:user</code>
<code>SEED_PASSWORD</code>	Your login password	One-time; remove after <code>seed:user</code>

6 Build & Database Setup

6.1 Create runtime directories

```
# As fms user – create data, logs, and backup directories
mkdir -p /home/fms/{data,logs,backups}
chmod 700 /home/fms/data /home/fms/backups
chmod 750 /home/fms/logs
```

6.2 Build the application

VAPID KEY IS RUNTIME CONFIG — NO SPECIAL BUILD STEP NEEDED

`NEXT_PUBLIC_VAPID_PUBLIC_KEY` is declared as `runtimeConfig.public.vapidPublicKey` in `nuxt.config.ts` and resolved at runtime via `useRuntimeConfig()`. Do **not** source `.env` before `npm run build`. PM2's `--update-env` at reload injects it automatically.

```
cd /home/fms/htdocs/fms.argontechs.dev
```

```
# Build Nitro server bundle → .output/server/index.mjs
npm run build
```

Verify the build artifact exists:

```
ls -lh .output/server/index.mjs
# Expected: file present, non-zero size
```

6.3 Run migrations and seed

```
# Load env so DATABASE_URL points to /home/fms/data/money.sqlite
set -a; . ./env; set +a

# Run Drizzle migrations (creates tables)
npm run db:migrate

# Seed recurring templates, debts, accounts, and goals (one-time only)
npm run db:seed

# Bootstrap the single login user from SEED_USERNAME / SEED_PASSWORD
npm run seed:user
```

Verify the database was created correctly:

```
sqlite3 /home/fms/data/money.sqlite "PRAGMA journal_mode;"
# Expected: wal

sqlite3 /home/fms/data/money.sqlite "PRAGMA foreign_keys;"
# Expected: 1

sqlite3 /home/fms/data/money.sqlite ".tables"
# Expected: accounts debts goals notifications_sent push_subscriptions
#           recurring_items sessions transactions users

sqlite3 /home/fms/data/money.sqlite "SELECT count(*) FROM recurring_items;"
# Expected: 16

sqlite3 /home/fms/data/money.sqlite "SELECT count(*) FROM users;"
# Expected: 1

sqlite3 /home/fms/data/money.sqlite "SELECT substr(password_hash,1,10) FROM users;"
# Expected: $argon2id$
```

REMOVE BOOTSTRAP CREDENTIALS AFTER SEEDING

Once `seed:user` succeeds, remove `SEED_USERNAME` and `SEED_PASSWORD` from `.env` — they are not needed at runtime and should not persist in the environment.

7 PM2 — Process Management & Reboot Persistence

The repo's `ecosystem.config.cjs` declares exactly **one** app (`money-fms`), fork mode, 1 instance. There is no second `money-scheduler` process — scheduling runs in-process inside `money-fms` . Running two apps would double-fire every notification and double-post every recurring transaction.

7.1 Start under PM2

```
cd /home/fms/htdocs/fms.argontechs.dev

pm2 start ecosystem.config.cjs
pm2 save
```

Verify:

```
pm2 list
# Expected: money-fms | online | fork | 1 instance

pm2 jlist | node -e "
  const a = JSON.parse(require('fs').readFileSync(0));
  const p = a.find(x => x.name === 'money-fms');
  console.log(p.pm2_env.status, p.pm2_env.exec_mode, p.pm2_env.instances);
"
# Expected: online fork 1
```

7.2 Reboot persistence via linger

LOGINCTL ENABLE-LINGER IS THE CRITICAL STEP

Without linger, the PM2 systemd unit is killed at session end and does not start at boot. The systemd unit is necessary but not sufficient — linger is what makes it survive reboot without an interactive login.

```
# Step 1 – Generate the startup command (run as fms user)
pm2 startup systemd -u fms --hp /home/fms
# This prints a sudo command – copy it exactly, then run it as ROOT:
# sudo env PATH=$PATH:/usr/bin /usr/local/bin/pm2 startup systemd -u fms --hp /home/fms

# Step 2 – Run the printed sudo command AS ROOT
# (SSH as root or use sudo)
sudo env PATH=$PATH:/usr/bin /usr/local/bin/pm2 startup systemd -u fms --hp /home/fms

# Step 3 – Enable linger (the line that actually makes it survive reboot)
sudo loginctl enable-linger fms

# Step 4 – Save the PM2 process list
pm2 save
```

Verify linger is enabled:


```
loginctl show-user fms --property=Linger
# Expected: Linger=yes
```

7.3 Verify after reboot (from a root shell)

MUST VERIFY FROM A ROOT SHELL AFTER REBOOT

SSHing as the site user masks a missing linger — the user session starts PM2 on login, making a broken setup appear to work. SSH as root after reboot to see the true state.

```
sudo reboot

# After the box comes back, SSH as ROOT:
pgrep -u fms -fa 'PM2|money-fms'
sudo -u fms pm2 list
# Expected: money-fms shows "online" WITHOUT any fms interactive login
```

7.4 ecosystem.config.cjs (reference)

```
// ONE app only – scheduling is in-process, not a second process
const cwd = '/home/fms/htdocs/fms.argontechs.dev'
module.exports = {
  apps: [{
    name:          'money-fms',
    cwd,
    script:        '.output/server/index.mjs',
    exec_mode:     'fork',
    instances:     1,
    env_file:      '.env',
    env:           { TZ: 'Asia/Kuala_Lumpur' },
    max_memory_restart: '400M',
    out_file:      '/home/fms/logs/money-fms-out.log',
    error_file:    '/home/fms/logs/money-fms-error.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
  }],
}
```

8 HTTPS & Reverse Proxy

8.1 Verify app answers on loopback first

```
curl -sI http://127.0.0.1:3000/
# Expected: HTTP/1.1 200 OK (or 302 to login)
# If this fails, fix PM2 before touching TLS
```

8.2 Issue the Let's Encrypt certificate

In CloudPanel: Sites → fms.argontechs.dev → SSL/TLS → New Let's Encrypt Certificate → Create and Install.

This enables auto HTTP→HTTPS redirect and auto-renewal.

Verify TLS and redirect:

```
curl -sI http://fms.argontechs.dev/ | grep -i '^location:'  
# Expected: Location: https://fms.argontechs.dev/  
  
curl -sI https://fms.argontechs.dev/ | head -1  
# Expected: HTTP/2 200 (or HTTP/2 302)
```

8.3 Required nginx vhost additions

In the CloudPanel vhost for `fms.argontechs.dev`, ensure the proxy `location` block contains these directives and add the internal-API deny block:

```
proxy_set_header X-Forwarded-Proto $scheme;  
proxy_set_header Host $host;  
proxy_set_header Upgrade $http_upgrade;  
proxy_set_header Connection "upgrade";  
  
# Block the watchdog endpoint from the public internet  
location ^~ /api/internal/ {  
    deny all;  
    return 404;  
}  
  
# Block dotfiles (prevents .env leakage)  
location ~ /\. {  
    deny all;  
    return 403;  
}
```

Reload nginx after any vhost change:

```
sudo nginx -t && sudo systemctl reload nginx  
# Expected: syntax is ok / test is successful
```

8.4 Verify security headers and blocks

```
curl -sI https://fms.argontechs.dev/ | grep -i 'strict-transport-security'  
# Expected: Strict-Transport-Security header present  
  
curl -sI https://fms.argontechs.dev/.env  
# Expected: 403  
  
curl -sI https://fms.argontechs.dev/api/internal/run-due  
# Expected: 404 (nginx deny block – endpoint is NOT publicly reachable)
```

SESSION COOKIE ATTRIBUTES ARE HARD-CODED IN APPLICATION CODE

`setCookie` does not auto-add `Secure`, and `X-Forwarded-Proto` is spoofable from the loopback port. The application sets `httpOnly:true, secure:true, sameSite:'lax', domain:'fms.argontechs.dev'` explicitly in code — not inferred from any header.

8.5 Verify the session cookie

```
curl -si https://fms.argontechs.dev/api/auth/login \  
  -X POST \  
  -H 'content-type: application/json' \  
  -d '{"username":"<your-username>","password":"<your-password>"}' \  
  | grep -i '^set-cookie:' \  
# Expected: Secure; HttpOnly; SameSite=Lax; Domain=fms.argontechs.dev
```

9 Habit-Engine Watchdog Cron

The **primary** dispatch path is the in-process Nitro scheduler (croner) running inside `money-fms`. Two tasks are registered in `nuxt.config.ts`:

- `notify-dispatch` — `* / 5 * * * *` — bill-due reminders and payday-save prompts
- `post-recurring` — `5 0 * * *` — auto-posts salary, bills, and card interest accrual

The OS-cron watchdog is **permanent standing insurance** for the in-process scheduler. It must NOT be removed once croner is confirmed working. If the Nitro scheduler ever fails to start (e.g. after a Nuxt major version bump), the watchdog keeps notifications firing.

9.1 Install the watchdog cron

Run as the `fms` site user. The cron sources `.env` so it picks up `NUXT_RUN_DUE_SECRET` from the same file as the app:

```
# First verify the endpoint responds correctly (app must be running)
set -a; . /home/fms/htdocs/fms.argontechs.dev/.env; set +a

# Test: no secret → 401
curl -s -o /dev/null -w '%{http_code}\n' \
  -X POST http://127.0.0.1:3000/api/internal/run-due
# Expected: 401

# Test: correct secret → 200
curl -s -o /dev/null -w '%{http_code}\n' \
  -X POST http://127.0.0.1:3000/api/internal/run-due \
  -H "x-run-due-secret: $NUXT_RUN_DUE_SECRET"
# Expected: 200

# Install the cron (idempotent – check first to avoid duplicates)
crontab -l 2>/dev/null | grep -q 'run-due-watchdog' || (
  ( crontab -l 2>/dev/null; \
    echo 'TZ=Asia/Kuala_Lumpur'; \
    echo "* / 5 * * * * set -a; . /home/fms/htdocs/fms.argontechs.dev/.env; set +a; curl -fsS \
-X POST -H \"x-run-due-secret: \$NUXT_RUN_DUE_SECRET\" http://127.0.0.1:3000/api/internal/r \
un-due >> /home/fms/logs/run-due.log 2>&1" \
  ) | crontab -
)

# Verify
crontab -l | grep run-due
# Expected: the */5 line is present
```

9.2 Endpoint security model

Layer	Protection
Nginx	<code>location /api/internal/ { deny all; return 404; }</code> — endpoint is never reachable from the public internet
Application	Constant-time comparison of <code>x-run-due-secret</code> header against <code>NUXT_RUN_DUE_SECRET</code> — returns 401 on mismatch
Network	Cron hits <code>http://127.0.0.1:3000</code> — loopback only, never the public hostname
Idempotency	<code>notifications_sent UNIQUE(kind, ref_id, scheduled_for)</code> — both croner and watchdog firing in the same window produce exactly one notification

9.3 Smoke test — verify croner fires in-process

```
# After deploy, wait ~5-7 minutes then check logs
pm2 logs money-fms --lines 200 --no-stream | grep notify-dispatch
# Expected: at least one line like:
#   [notify-dispatch] 2026-06-19T01:05:00.000Z sent=0 skipped=2

# Check no duplicate notifications were sent
sqlite3 /home/fms/data/money.sqlite \
  "SELECT kind, ref_id, scheduled_for, count(*) c
  FROM notifications_sent
  GROUP BY kind, ref_id, scheduled_for
  HAVING c > 1;"
# Expected: no rows
```

IF CRONER DOES NOT FIRE AFTER 10 MINUTES

The in-process scheduler did not start under the `node-server` preset. The OS-cron watchdog is then the **sole trigger** — confirm it is installed and firing before trusting the habit engine. Record the result in your deployment notes.

10 Backups to Google Drive

NEVER USE CP ON A WAL DATABASE

SQLite in WAL mode uses a `-wal` sidecar file. A bare `cp money.sqlite` copies the main file without `-wal`, silently losing recently committed transactions and risking a corrupt restore. Always use `sqlite3 ".backup"` for a consistent hot snapshot.

10.1 One-time rclone setup for Google Drive

```
# Install rclone if not present
apt install rclone # or: curl https://rclone.org/install.sh | sudo bash

# Configure the named Google Drive remote (interactive OAuth)
rclone config
# → New remote → name: gdrive-money → type: drive → follow OAuth prompts

# Lock the rclone config
chmod 600 ~/.config/rclone/rclone.conf

# Verify auth works
rclone listremotes | grep gdrive-money:
rclone about gdrive-money:
# Expected: gdrive-money: listed; storage quota JSON returned
```

10.2 backup-db.sh — full script

```

#!/usr/bin/env bash
# /home/fms/bin/backup-db.sh
# WAL-safe hot snapshot → gzip → rclone to Google Drive BEFORE local prune
set -euo pipefail

DB="/home/fms/data/money.sqlite"
DEST="/home/fms/backups"
REMOTE="${RCLONE_REMOTE:-gdrive-money}:money-fms-backups"
STAMP=$(date +%Y%m%d-%H%M%S)
OUT="$DEST/money-$STAMP.sqlite"
LOG="$DEST/backup.log"

alert() { echo "[$(date -u +%FT%TZ)] BACKUP-ALERT: $" | tee -a "$LOG" >&2; }

# 1. Consistent hot snapshot (NEVER cp - WAL-safe)
sqlite3 "$DB" ".backup '$OUT'"
gzip "$OUT"
ARCHIVE="$OUT.gz"
chmod 600 "$ARCHIVE"

# 2. Size sanity check before trusting the archive
SIZE=$(stat -c%s "$ARCHIVE")
if [ "$SIZE" -lt 1024 ]; then
    alert "archive suspiciously small (${SIZE} bytes) – aborting prune"
    exit 1
fi

# 3. Upload off-box FIRST (non-fatal: alert but do not let a Drive
#    outage kill local backup) – never prune before off-box copy succeeds
if rclone copy "$ARCHIVE" "$REMOTE/" --quiet; then
    if ! rclone lsf "$REMOTE/" | grep -q "${basename "$ARCHIVE"}"; then
        alert "rclone success reported but file not found on remote – NOT pruning"
        exit 1
    fi
    echo "[$(date -u +%FT%TZ)] off-box OK: $(basename "$ARCHIVE") (${SIZE}B)" \
    >> "$LOG"
else
    alert "rclone to $REMOTE FAILED – keeping ALL local copies, skipping prune"
    exit 1
fi

# 4. Prune ONLY after confirmed off-box copy
# Local: keep 14 days
find "$DEST" -name 'money-*.sqlite.gz' -mtime +14 -delete
# Off-box: keep 60 days
rclone delete "$REMOTE/" --min-age 60d --quiet \
|| alert "off-box prune (60d) failed (non-fatal)"

echo "[$(date -u +%FT%TZ)] backup complete" >> "$LOG"

```

10.3 restore-verify.sh — full script

A backup never restored is not a backup. This script runs nightly after the backup to confirm the archive is valid.

```
#!/usr/bin/env bash
# /home/fms/bin/restore-verify.sh
set -euo pipefail

DEST="/home/fms/backups"
LOG="$DEST/restore-verify.log"
SCRATCH="$(mktemp -d)"
trap 'rm -rf "$SCRATCH"' EXIT

alert() { echo "[$(date -u +%FT%TZ)] RESTORE-VERIFY-ALERT: $" \
  | tee -a "$LOG" >&2; }

LATEST="$(ls -t "$DEST"/money-*.sqlite.gz 2>/dev/null | head -1)"
[ -n "$LATEST" ] || { alert "no backup archive found"; exit 1; }
zcat "$LATEST" > "$SCRATCH/restore.sqlite"

# 1. Structural integrity check
IC="$(sqlite3 "$SCRATCH/restore.sqlite" 'PRAGMA integrity_check;')"
[ "$IC" = "ok" ] || { alert "integrity_check FAILED on $LATEST: $IC"; exit 1; }

# 2. Row-count sanity (seeded data guarantees these are never zero)
RC="$(sqlite3 "$SCRATCH/restore.sqlite" 'SELECT count(*) FROM recurring_items;')"
[ "$RC" -ge 16 ] || { alert "recurring_items count too low ($RC)"; exit 1; }
UC="$(sqlite3 "$SCRATCH/restore.sqlite" 'SELECT count(*) FROM users;')"
[ "$UC" -eq 1 ] || { alert "users count != 1 ($UC)"; exit 1; }

# 3. Latest-timestamp freshness (newest transaction within 8 days)
TXN="$(sqlite3 "$SCRATCH/restore.sqlite" 'SELECT count(*) FROM transactions;')"
if [ "$TXN" -gt 0 ]; then
  MAXC="$(sqlite3 "$SCRATCH/restore.sqlite" \
    'SELECT max(created_at) FROM transactions;')"
  NOW_MS=$(( $(date +%s) * 1000 ))
  AGE_DAYS=$(( (NOW_MS - MAXC) / 86400000 ))
  [ "$AGE_DAYS" -le 8 ] \
    || alert "newest transaction is ${AGE_DAYS}d old (stale?)"
fi

echo "[$(date -u +%FT%TZ)] restore-verify OK: $LATEST \
(integrity ok, recurring=$RC, users=$UC, txns=$TXN)" >> "$LOG"
```

10.4 Install scripts and cron jobs


```

mkdir -p /home/fms/bin

# Install scripts
cp bin/backup-db.sh /home/fms/bin/
cp bin/restore-verify.sh /home/fms/bin/
chmod 700 /home/fms/bin/backup-db.sh /home/fms/bin/restore-verify.sh

# Run backup once to verify everything works end-to-end
set -a; . /home/fms/htdocs/fms.argontechs.dev/.env; set +a
/home/fms/bin/backup-db.sh

# Verify the archive
ls -lh /home/fms/backups/money-*.sqlite.gz | tail -1
stat -c '%a' "$(ls -t /home/fms/backups/money-*.sqlite.gz | head -1)"
# Expected: file present, permissions 600

# Verify the archive is on Google Drive
rclone lsf gdrive-money:money-fms-backups/ | tail -1

# Run restore-verify manually once
/home/fms/bin/restore-verify.sh
tail -1 /home/fms/backups/restore-verify.log
# Expected: restore-verify OK: ... (integrity ok, recurring=16, users=1)

# Add cron jobs (backup at 03:15, verify at 03:45 MYT)
( crontab -l 2>/dev/null | grep -v 'backup-db\|restore-verify';
  echo "TZ=Asia/Kuala_Lumpur";
  echo "15 3 * * * set -a; . /home/fms/htdocs/fms.argontechs.dev/.env; set +a; /home/fms/bin/backup-db.sh >> /home/fms/backups/backup.log 2>&1";
  echo "45 3 * * * /home/fms/bin/restore-verify.sh >> /home/fms/backups/restore-verify.log 2>&1"
) | crontab -

# Verify crontab
crontab -l
# Expected: TZ line + backup-db line at 03:15 + restore-verify at 03:45

```

11 Deploy & Update Routine

11.1 deploy.sh — full script

```
#!/usr/bin/env bash
# /home/fms/bin/deploy.sh
# Pull → install → migrate → build → reload (zero-downtime)
set -euo pipefail

cd /home/fms/htdocs/fms.argontechs.dev

git pull --ff-only

npm ci

# Run migrations BEFORE reload – new code never hits the old schema
npm run db:migrate

# VAPID public key is RUNTIME config (useRuntimeConfig) – do NOT source
# .env before build. pm2 reload --update-env injects it at runtime.
npm run build

pm2 reload ecosystem.config.cjs --update-env
pm2 save

echo "Deployed money-fms @ $(git rev-parse --short HEAD)"
```

11.2 Install and use deploy.sh

```
mkdir -p /home/fms/bin
cp bin/deploy.sh /home/fms/bin/deploy.sh
chmod 700 /home/fms/bin/deploy.sh

# Run a deploy
/home/fms/bin/deploy.sh
# Expected final line: Deployed money-fms @ <sha>

# Verify PM2 is still online after reload
pm2 list
# Expected: money-fms | online | fork | 1 instance
```

11.3 Rollback procedure

The SQLite database lives at `/home/fms/data/money.sqlite` — outside the docroot — and is untouched by a code rollback.

```
# Identify the last known-good commit
git log --oneline -10

# Roll back to a specific SHA
git reset --hard <sha>

npm ci
npm run db:migrate    # safe - Drizzle only applies new migrations
npm run build

pm2 reload ecosystem.config.cjs --update-env
pm2 save

echo "Rolled back to $(git rev-parse --short HEAD)"
```

SCHEMA ROLLBACK

If you must roll back a migration, restore from the most recent backup (`/home/fms/backups/`) to a staging path, verify, then swap. Never manually alter a production SQLite schema — use Drizzle migrations.

12 Verification Checklist

Run this checklist from a **root shell** after a fresh `sudo reboot` to confirm the full deployment is healthy.

12.1 Infrastructure

- ☐ `dig +short fms.argontechs.dev` returns the VPS public IP
- ☐ `curl -sI http://fms.argontechs.dev/ | grep -i '^location:'` → `https://fms.argontechs.dev/`
- ☐ `curl -sI https://fms.argontechs.dev/ | head -1` → `HTTP/2 200` or `302`
- ☐ `curl -sI https://fms.argontechs.dev/ | grep -i 'strict-transport'` → HSTS header present
- ☐ `curl -sI https://fms.argontechs.dev/.env` → `403`
- ☐ `curl -sI https://fms.argontechs.dev/api/internal/run-due` → `404`

12.2 Process health

- ☐ `sudo -u fms pm2 list` → `money-fms | online | fork | 1` (without interactive fms login)
- ☐ `logintctl show-user fms --property=Linger` → `Linger=yes`
- ☐ `crontab -u fms -l` → three jobs: `backup-db.sh 03:15`, `restore-verify.sh 03:45`, `run-due watchdog`
*/5

12.3 Database

- ☐ `sqlite3 /home/fms/data/money.sqlite "PRAGMA journal_mode;"` → `wal`
- ☐ `sqlite3 /home/fms/data/money.sqlite "PRAGMA integrity_check;"` → `ok`
- ☐ All 9 v1 tables present: `accounts` `debts` `goals` `notifications_sent` `push_subscriptions` `recurring_items` `sessions` `transactions` `users`
- ☐ `recurring_items` count = 16 · `users` count = 1 · password_hash starts with `$argon2id$`

12.4 Backups

- ❑ A `money-*.sqlite.gz` file exists in `/home/fms/backups/` with permissions `600`
- ❑ The same file appears in `rcclone lsf gdrive-money:money-fms-backups/`
- ❑ `tail -1 /home/fms/backups/restore-verify.log` → `restore-verify OK: ...`

12.5 Authentication

- ❑ Login at `https://fms.argontechs.dev/login` succeeds with your credentials
- ❑ Dashboard loads and shows the Safe-to-Spend hero number
- ❑ Unauthenticated `POST /api/push/subscribe` returns `401`
- ❑ Login `Set-Cookie` contains `Secure; HttpOnly; SameSite=Lax; Domain=fms.argontechs.dev`

12.6 Habit engine

- ❑ `pm2 logs money-fms --nostream | grep notify-dispatch` → at least one fire line within the last 5 minutes
- ❑ No duplicate rows in `notifications_sent` — run GROUP BY check (§9.3)
- ❑ On iPhone: open in Safari → Share → **Add to Home Screen** → open installed app → tap "Turn on reminders" → accept permission → "Reminders are working" canary notification arrives
- ❑ iPhone app shows the standalone health signal (not the "Add to Home Screen" banner)

13 First-Run Real-World Actions

The app models your financial position but cannot move money. Once the dashboard loads and you see your Safe-to-Spend number, take these actions in the real world to align reality with the model. These are ordered by impact.

GENERAL INFORMATION — NOT LICENSED FINANCIAL ADVICE

Verify current rates, terms, and campaign availability with each provider before acting.

Priority	Action	Why
1 — URGENT	Ring-fence the ~RM1,000 cash on hand into the RYT Bank "Emergency Fund" Save Pocket (PIDM-insured deposit, instant, ~3% p.a.). Log the transfer in the app. The starter buffer is ~75% already covered by existing cash — this is consolidation, not net-new saving.	Breaks the RM0-buffer → 18%-card loop on day one. First habit win.
2 — URGENT	Ask your card issuer (the bank that issued your credit card) about a balance installment / conversion plan for the ~RM7,400 balance — accept only if the EIR clearly beats 18%. If declined, compare open-market 0% BT cards. Update <code>debts.bt_status</code> in the app.	A 0% conversion saves ~RM444–490 in interest over the kill-card window. Every ringgit of surplus goes to principal, not interest.
3 — THIS MONTH	Contact Great Eastern about the "Great Wealth Enhancer" ILP (commenced 18 May 2026, ~1 month in, charged to the credit card ••0509): (a) confirm premium term (5 vs 10 years), (b) check cooling-off / free-look status, (c) request a premium holiday or paid-up option. Stop the auto-charge to the card immediately. The critical-illness plan (RM350/mo, due 27th) is separate — leave it untouched.	You are borrowing at 18% to fund a ~5% plan. Redirecting RM350/mo cuts card-payoff time by weeks and removes a guaranteed annual loss.
4 — THIS MONTH	Flip card-funded auto-debits (Digi, Unifi, gym, subscriptions) to debit / bank account. Stop charging daily living to the credit card while it carries a balance. Update the <code>funding_account_id</code> on each recurring template in the app.	The model assumes card balance only falls. If auto-debits keep charging, the balance re-pollutes and the card-free date projection is wrong.

14 Troubleshooting

14.1 Login page redirects to HTTP or cookie is rejected

SYMPTOM

Browser shows a mixed-content warning, login fails, or the session cookie is silently dropped.

Cause: The session cookie is marked `Secure` in code — it requires HTTPS. If you are hitting HTTP, the browser will not send the cookie and every request appears unauthenticated. Similarly, if the domain in the cookie does not match the request origin, the browser drops it.

Fix: Confirm the Let's Encrypt cert is installed (§8.2) and HTTP redirects to HTTPS (`curl -sI http://fms.argontechs.dev/ | grep location`). Clear browser cookies and retry over HTTPS only.

14.2 Push notifications are silent on iPhone

SYMPTOM

The in-app "Turn on reminders" flow completes without error, the subscription row appears in the database, but no notifications ever arrive on the iPhone.

Most common cause: app is not installed to the Home Screen. iOS Web Push requires the PWA to be running in standalone mode (installed, not in a Safari tab). A Safari tab silently accepts the subscription but never delivers push.

Steps to fix:

1. On iPhone, open `https://fms.argontechs.dev` in Safari.
2. Tap the Share button → "Add to Home Screen" → Add.
3. Open the app from the Home Screen icon (not from Safari).
4. The app should show the "standalone" health indicator, not the "Add to Home Screen" banner.
5. Tap "Turn on reminders" again — the prior subscription (created from Safari) is no longer valid for delivery. A new subscription from standalone mode will work.
6. Verify the new subscription row: `sqlite3 ../money.sqlite "SELECT id, failed_at FROM push_subscriptions ORDER BY id DESC LIMIT 3;"` — the newest row should have `failed_at` NULL.
7. If `failed_at` is set, the VAPID key mismatch is the next candidate: confirm `NEXT_PUBLIC_VAPID_PUBLIC_KEY` in PM2 env exactly matches the value used to generate `NEXT_VAPID_PRIVATE_KEY`.

14.3 Scheduler not firing — no notify-dispatch lines in logs

Diagnosis:

```
pm2 logs money-fms --lines 300 --nostream | grep -iE 'task|cron|notify|scheduler'
# If empty after 10 minutes, croner did not start under node-server preset
```

Immediate fix: Confirm the OS-cron watchdog (\$9) is installed and firing. Check `/home/fms/logs/run-due.log` for recent `http=200` lines.

Root cause check: Confirm `nuxt.config.ts` has `nitro.preset: 'node-server'` and `nitro.experimental.tasks: true`. Confirm task file names are flat (`notify-dispatch.ts` , NOT `notify/dispatch.ts`) and registered names match exactly (`'notify-dispatch'` , NOT `'notify:dispatch'`). A colon in the task name resolves to a nested directory path and silently never fires.

14.4 Build fails with environment-related errors

Do not source `.env` before `npm run build`. The VAPID public key is runtime config — the build does not need it. If you see errors like `NEXT_PUBLIC_VAPID_PUBLIC_KEY is not defined at build time`, this is expected and correct; the key is injected by PM2 at runtime via `--update-env`.

14.5 Database locked or WAL errors

The app runs as a single PM2 fork (1 instance). If you see `SQLITE_BUSY` or WAL-related errors, confirm there is only one process writing:

```
pm2 list | grep money-fms
# Expected: exactly 1 row, exec_mode: fork, instances: 1

# Also confirm no stale backup process is holding a file lock
fuser /home/fms/data/money.sqlite
```

14.6 Reboot — PM2 does not start automatically

The most common cause is missing linger. Check:

```
loginctl show-user fms --property=Linger
# Expected: Linger=yes
# If no: sudo loginctl enable-linger fms && pm2 save && sudo reboot
```

Also confirm the systemd unit exists: `systemctl status pm2-fms --no-pager | head -3`

14.7 Backup file suspiciously small or Google Drive upload fails

Check `/home/fms/backups/backup.log` for `BACKUP-ALERT` lines. Common causes: `sqlite3` not installed, `RCLONE_REMOTE` env var not set (ensure `.env` is sourced in the cron line), or Google Drive OAuth token expired (re-run `rcclone config reconnect gdrive-money:`).